

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

ABSTRACT

CheckMate presents a comprehensive, AI-driven plagiarism and idea originality detection system designed to address critical challenges in academic integrity, such as direct copying, advanced paraphrasing, and the increasing misuse of AI-generated content. CheckMate is developed as a standalone, lightweight web application utilizing a Python and Flask backend architecture. It utilizes Jinja2 for frontend template rendering and relies on an efficient local file system and in-memory caching strategy for data storage, ensuring rapid deployment and portability without the overhead of external database servers.

The core innovation of CheckMate lies in the seamless integration of multiple Natural Language Processing and Deep Learning models for comprehensive text analysis, augmented by built-in dataset management tools:

1. Copied Content Detection Model (TF-IDF with Cosine Similarity):

This model represents documents as weighted term vectors and computes similarity scores to detect directly copied content. It is optimized for fast comparison against a locally managed reference corpus stored as text and CSV files, serving as the highly efficient baseline plagiarism detection mechanism.

2. Paraphrase Detection Model (SBERT - Sentence-BERT):

Fine-tuned for semantic similarity analysis, this model captures contextual meaning beyond surface-level text matching. It effectively identifies paraphrased content where wording and sentence structure are altered while preserving the original meaning, utilizing pre-computed semantic pairs.

3. AI-Generated Content Detection Model:

This model analyzes linguistic patterns, coherence, and probability distributions to distinguish between human-written and AI-generated text. It assigns an AI-likelihood score, helping faculty identify submissions influenced by generative AI tools.

4. Originality Scoring Engine with Dynamic Weights:

Outputs from all detection modules are combined using a weighted scoring mechanism to generate a comprehensive originality score. A key innovation in this implementation allows faculty members to dynamically adjust the influence of each model (e.g., 40% TF-IDF, 30% SBERT, 30% AI) directly from their dashboard.

5. Integrated Dataset Management and arXiv Importer:

The system features an automated, multi-threaded arXiv paper importer. Faculty can query topics, and the system will automatically download PDFs, extract text in the background, rebuild SBERT datasets, and utilize a keyword-based engine to auto-categorize reference papers into domains like AI/ML, Medical & Healthcare, and Computer Vision.

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

The CheckMate platform includes role-based dashboards for students and faculty, submission history tracking, and dynamically generated, color-coded originality reports using the FPDF library.

LIST OF FIGURES

Fig. 4.1 Technologies Used

Fig. 4.2 System Methodology (Flow Diagram)

Fig. 5.1 Copied Content Detection (TF-IDF Model)

Fig. 5.5.i Student Dashboard Interface

Fig. 5.5.ii Faculty Dashboard Interface

LIST OF TABLES

Table 2.1 CheckMate System Scope and Benefits

Table 2.2 User Classes and Characteristics

Table 3.1 Operating Environment Details

Table 3.2 Non-Functional Requirements

Table 3.3 Hardware Requirements

Table 3.4 Software Requirements

Table 4.1 Technology Integration Details

Table 4.2 AI & NLP Model Integration Details

Table 5.4 Interpretation of Similarity Detection Outcomes

CHAPTER 1: INTRODUCTION

Academic integrity is a fundamental principle of education and research. However, the widespread availability of online resources and AI-based content generation tools has significantly increased the risk of plagiarism in academic submissions. Manual plagiarism checking is time-consuming and prone to human error, making automated systems essential. CheckMate aims to provide an efficient, accurate, and scalable Python-based solution for plagiarism and AI-content detection in academic environments.

1.1 Purpose

The purpose of this project is to design and implement an end-to-end intelligent system named CheckMate, aimed at ensuring academic originality and integrity in educational institutions. With the rapid increase in digital content availability and the widespread use of generative Artificial Intelligence tools, traditional plagiarism detection methods have become insufficient. Many existing systems fail to detect paraphrased or AI-generated content accurately.

CheckMate aims to assist both students and faculty by providing a reliable, automated mechanism for detecting copied, paraphrased, and AI-generated text. The system focuses on transparency by highlighting problematic sections, explaining the nature of similarity detected, and providing a beautifully formatted, downloadable PDF report using FPDF. It provides actionable feedback that helps students improve the originality of their work.

1.2 Project Scope

The scope of the CheckMate project includes the development of a standalone web-based plagiarism and originality detection platform built using the Flask micro-framework. The system analyzes PDF documents submitted by students and faculty.

The platform provides role-based access. Faculty members have elevated privileges, allowing them to adjust similarity scoring weights, manually manage dataset files, and trigger background tasks to import new research papers directly from arXiv. Students are limited to uploading documents and viewing or downloading detailed analysis reports. The project utilizes local file structures (userinput, datasets, reference_papers) and in-memory dictionaries for session tracking, ensuring it remains highly portable and independent of complex external database dependencies.

1.3 Literature Survey

The literature survey reveals that traditional plagiarism detection systems primarily rely on string matching and keyword-based techniques. While these methods are effective for detecting exact copies, they fail when the text is paraphrased or semantically altered.

Recent studies emphasize the use of TF-IDF combined with cosine similarity as a baseline approach for identifying copied text. However, advancements in NLP have led to the adoption of transformer-based

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

models such as BERT and SBERT, which capture contextual meaning and sentence-level similarity more effectively. Additionally, existing commercial tools are often expensive, closed-source, and lack transparency. This motivates the development of CheckMate as a transparent, Python-native solution where faculty can directly oversee and expand the datasets being used to evaluate student submissions.

CHAPTER 2: OVERALL DESCRIPTION

2.1 Project Perspective

CheckMate is designed as a standalone web application that replaces manual and semi-automated plagiarism checking methods. It is architected as a monolithic Flask application. By combining the web server, the AI processing logic, and data storage into a cohesive Python environment, the system is exceptionally easy to deploy on institutional servers or local machines.

2.2 Project Functions

CheckMate performs several core functions essential for academic originality verification:

- Document Upload & Preprocessing: PDFs are uploaded into a local userinput directory, where their text is extracted and tokenized.
- Plagiarism & Similarity Detection: TF-IDF and SBERT models cross-reference extracted text against locally stored .txt and .csv datasets.
- Dynamic Originality Scoring: Configurable weighting system (e.g., TF-IDF, SBERT, AI probabilities) allows faculty to dictate the strictness of the originality score.
- ArXiv Dataset Importer: A specialized background thread that queries arXiv, downloads relevant PDFs, extracts their text, and rebuilds the SBERT semantic pairs dataset on the fly without interrupting the web interface.
- Auto-Categorization: Automatically categorizes reference materials into specific domains (AI/ML, Mathematics, Medical) using targeted keyword analysis.
- Report Generation: Generates comprehensive PDF reports with color-coded progress bars and sentence-by-sentence analysis using the FPDF library.

2.3 User Classes and Characteristics

The system supports two primary user classes: Students and Faculty Members. Each user class interacts with the system differently through session-based authentication.

- Students primarily interact with the document upload tools. They view their on-screen results and download PDF reports. Their submission history is tracked for the duration of the session.
- Faculty members have an advanced dashboard. They can adjust the global MODEL_WEIGHTS, oversee the categories of reference papers, manually upload new data files to the tfidf or sbert folders, and run the arXiv importer tool to expand the system's knowledge base.

2.4 Operating Environment

CheckMate operates in a web-based environment accessible through standard browsers (Chrome, Edge). The entire backend is powered by Python and Flask. The system is designed to run on local file systems without requiring MongoDB or SQL databases. Temporary files, reference PDFs, and serialized datasets are stored directly in structured folders relative to the application's root directory, ensuring maximum portability.

CHAPTER 3: REQUIREMENTS

3.1 Functional Requirements

- FR1: Role-based login system for Students and Faculty.
- FR2: Support for PDF document uploads and automated text extraction.
- FR3: Copied content detection using TF-IDF against a local corpus.
- FR4: Paraphrase detection via Sentence-BERT (SBERT).
- FR5: AI content detection using machine learning classifiers.
- FR6: Background threading for arXiv dataset importing.
- FR7: Automated keyword-based paper categorization.
- FR8: Dynamic generation of downloadable PDF originality reports.

3.2 Non-Functional Requirements

- Performance: AI inference must execute within acceptable limits, and dataset imports must run asynchronously to prevent UI blocking.
- Scalability: The modular design must allow new reference papers to be added via the UI without requiring system restarts.
- Security: Secure file handling using `werkzeug.utils.secure_filename` to prevent path traversal attacks during file uploads.
- Portability: The system must rely entirely on local storage structures and Python-native libraries to simplify deployment.

3.4 Software Requirements

Software requirements are exclusively centered around the Python ecosystem:

- Backend Framework: Python 3.x, Flask, Werkzeug
- AI & NLP Processing: Scikit-learn, Sentence-Transformers, NLTK, Pandas
- Report Generation: FPDF
- Frontend UI: HTML, CSS, Jinja2 Templates
- Data Storage: Local File System (JSON, CSV, PDF, TXT)
- Browser: Chrome / Edge / Firefox

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The CheckMate system follows a streamlined, monolithic architecture based on the Flask framework.

- Presentation Layer (Frontend): Developed using HTML, CSS, and Jinja2 templates, served directly by Flask. It handles user interaction, file selection, and asynchronous polling for background task statuses.
- Application Layer (Backend): Flask routing handles user sessions, form processing, and triggers background threads.
- AI Processing Layer: Contains independent Python modules for TF-IDF vectorization, SBERT semantic matching, and AI classification.
- Data Storage Layer: Eliminates the need for external database servers by utilizing targeted directories (datasets/tfidf, datasets/sbert, reference_papers) and JSON files for categorization logic.

4.3 Module Description

The User Management module handles session-based authentication and role enforcement. The Document Processing module uses PDF extraction tools to clean and tokenize text. The Similarity Detection module applies mathematical models against local corpora. The Dataset Expansion module handles arXiv web scraping, downloading, and background corpus rebuilding. The Report Generation module utilizes FPDF to compile complex analytical data into visual documents.

4.4 AI Model Integration

- Copied Content Detection: Uses TF-IDF and Bag-of-words vectorization against locally stored .txt files. Computes cosine similarity to generate a Similarity Percentage indicating direct textual overlap.
- Paraphrase Detection: Employs pre-trained SBERT Transformers. Produces dense sentence embeddings and compares them against the sbert_pairs.csv dataset. Generates a Semantic Similarity Score highlighting paraphrased content.
- AI-Generated Content Detection: Fine-tuned linguistic classifier. Generates an AI Probability Score indicating the likelihood of AI-generated text.
- Originality Scoring Engine: Rule-based weighted aggregation logic (customizable via the Faculty Dashboard) that calculates the Final Originality Percentage and determines the Pass/Fail verdict.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Copied Content Detection Model Implementation Details

The copied content detection module was implemented using Python with the scikit-learn library. This feature utilizes the Term Frequency-Inverse Document Frequency (TF-IDF) vectorization technique combined with cosine similarity to identify direct textual overlap. TF-IDF transforms textual data into weighted numerical vectors, allowing effective comparison of term importance across documents.

During implementation, documents are preprocessed through tokenization, lowercasing, and stop-word removal. The TF-IDF vectorizer is trained on the fly using the local datasets/tfidf directory containing .txt files of academic documents. Cosine similarity is computed between the submitted document and this local corpus.

5.2 Paraphrase Detection Model Implementation

The paraphrase detection module employs the Sentence-BERT (SBERT) model for semantic similarity analysis. This model captures contextual meaning beyond surface-level text matching.

The implementation involves segmenting documents into sentences and encoding them into dense vector representations. Pairwise cosine similarity is computed between these embeddings and the local sbert_pairs.csv file. Because this dataset can become outdated, CheckMate implements an automated rebuilding process that regenerates the sentence pairs whenever new documents are ingested by the system.

5.3 AI-Generated Content Detection Implementation

To address the increasing use of generative AI tools, CheckMate integrates an AI content detection script that processes document segments and assigns an AI-likelihood score. The detection logic is seamlessly integrated directly into the Python backend, allowing near real-time processing alongside the plagiarism checks.

5.4 Originality Scoring Engine Implementation

The originality scoring engine combines the outputs of the detection modules. A significant architectural advantage of this implementation is dynamic configurability. Using the Faculty Dashboard, administrators can adjust a global MODEL_WEIGHTS dictionary.

Scoring Logic (Configurable Defaults):

- Copied Content (TF-IDF) Weight: 40%
- Paraphrased Content (SBERT) Weight: 30%
- AI Content Weight: 30%

5.5 Application Features Implemented

CHECKMATE: SMART PLAGIARISM & AI DETECTOR

The user-facing features implemented in CheckMate include:

- Student Dashboard: Allows PDF document upload, immediate on-screen originality viewing, and tracks submission history via an in-memory session cache.
- Faculty Dashboard: Provides access to advanced controls, allowing faculty to configure similarity thresholds and manually manage dataset files through the UI.
- arXiv Dataset Importer: An asynchronous background worker that searches the arXiv database based on faculty queries. It automatically downloads PDFs, saves reference copies, extracts text for TF-IDF, and rebuilds the SBERT CSV dataset-all while providing live log updates to the frontend via an API endpoint.
- Auto-Categorization System: A keyword-based heuristic engine that automatically scans the titles and filenames of downloaded reference papers, organizing them into predefined categories like 'Medical & Healthcare' and 'Computer Engineering', saving the mappings to a JSON configuration file.
- Dynamic PDF Report Generation: Utilizing the Python FPDF library, the system programmatically generates beautiful, downloadable originality reports. These reports feature color-coded verdict banners, visual progress bars, and a line-by-line breakdown where sentences are tagged as 'Original', 'Copied', 'Paraphrased', or 'AI Generated'.